Can a neural network learn the remainder?

Modulo 7, data scale, and generalisation beyond the training range

Antreas Antoniou
antreas.io · Code, measurements and editable report

Working research report · 15 October 2026 · Version 0.2 draft
Completed small-data screen; dated 560-of-576-fit validation snapshot.
Independent report using the NeurIPS preprint template; not a NeurIPS submission.

Abstract

Modulo 7 has a short exact solution, but a network trained on examples must learn how to represent and use its periodic structure. We study this problem with scalar and binary inputs, 16 architecture variants and a target of roughly 32,768 learned parameters. In a completed short-budget screen, a shallow conditional hypernetwork reaches **94.56% ± 2.12%** test accuracy within the training range with 65,536 examples, but **11.94% ± 2.39%** outside it. Smaller training sets leave that configuration near chance; a deeper generator has strongly mixed seed outcomes. A separate observed-bit diagnostic improves some outside-range results without making them consistent. We then examine a larger experiment with one million unique training integers and ten passes per fit. Its dated snapshot contains **560 of 576 fits**, including the complete 288-fit binary grid. At the checkpoint chosen by within-range validation loss, 14 binary fits reach at least 99% within-range accuracy and seven reach at least 99% on both validation ranges. These outcomes occur in residual MLPs and conditional hypernetworks, but no fixed learning-rate/decay setting reaches 99% on both ranges across all three seeds. The 272 completed scalar fits retain cross-entropy close to uniform prediction. The follow-up's final selection and test phase remain pending. Throughout, ± denotes sample standard deviation over three optimisation seeds on one fixed dataset, not a confidence interval.

1 A small function with a useful sting

Take an integer, divide by seven, keep the remainder. The labels are $0, 1, 2, 3, 4, 5, 6, 0, 1, \ldots$. There is almost nothing to the program. But if all we give a network is a collection of input–output pairs, the useful structure has to come from somewhere: the input representation, the architecture, or what optimisation manages to build out of the examples.

The experiment separates two abilities. A model can be quite good at filling gaps between the numbers it has seen and still have no dependable way to continue the pattern. So we ask two separate questions. Can it predict *unseen values within the training range*? And does that prediction survive when we move *outside the range*? Calling both of those "generalisation" and reporting one number would throw away much of what makes this experiment interesting.

The original screen also included sphere volume, multiplication, a chirp and a pairwise-distance task. Sphere volume is a smooth cubic control, so "nonlinear" should not be mistaken for "intrinsically difficult." A fitted cubic solves that control to numerical precision. This report concentrates on modulo, where the representation and data-size effects were particularly clear. The repository retains the complete measurements for the other tasks too.

2 What the models actually receive

The target is a seven-way classification problem, $y = n \bmod 7$, trained with cross-entropy. We use two encodings of the same integers. The scalar version maps the within-range interval to $[-1, 1]$. The binary version supplies each bit as -1 or $+1$. Neither supplies the remainder or hand-designed periodic features.

These are very different starting points. In the scalar representation, moving one integer along the line usually changes the answer. In binary, the bits expose a compositional route to the answer. If $n = \sum_k b_k 2^k$, then

$$n \bmod 7 = \left(\sum_k b_k (2^k \bmod 7) \right) \bmod 7, \qquad 2^k \bmod 7 = 1, 2, 4, 1, 2, 4, \ldots .$$

That identity explains an available structure; we do not build it into the learned models. The exact modulo operation is the label oracle, not a learned baseline.

2.1 Architecture is more than the number of layers

The plain MLPs have 3, 20 or 50 linear layers, including the output, with GELU between hidden layers. Residual 20- and 50-layer MLPs provide a control for making depth trainable. Transformer variants have 2, 8 or 12 pre-normalised blocks, four attention heads, a learned position for each numeric feature and a classification token. Each bit becomes a token; scalar input gives just one numeric token.

The relational variants take 2, 8 or 12 rounds of pairwise messages. A learned two-layer function processes every ordered pair, including self-pairs; messages are averaged and added to the receiving representation. Each round has its own parameters. This is a small adaptation of the relational-network idea [2], not a replication of its original experiments. A scalar input gives this module very little relational structure to work with.

The conditional hypernetworks have the form the experiment was designed to test:

$$\hat{y} = f(x, w), \qquad w = g(x; \theta).$$

Only θ is independently learned. For each example, g emits *all* weights and biases of f; f then processes that same example. Gradients pass through both computations, but f owns no stored trainable parameters. Generating one network's weights with another is the HyperNetworks formulation [1]; here generation is conditioned directly on the current input. We vary g's depth over 2, 8 and 12 at f depth 3, then vary f over 1, 3 and 5 at g depth 8. Hidden layers in f have width eight.

Widths are chosen to approach 32,768 learned parameters. That makes deeper models narrower. It also leaves discrete differences in parameter count, especially for four-head transformers. Generated coefficients are counted separately from learned parameters. We are comparing small models with roughly matched parameter counts; we are not matching FLOPs or claiming every architecture receives its best recipe.

2.2 The completed screen

We use 256, 4,096 and 65,536 nested training examples, seeds 101/202/303, and learning rates 3×10^{-4} and 3×10^{-3}. AdamW [3] uses zero weight decay, default $\beta = (0.9, 0.999)$ and $\epsilon = 10^{-8}$, no dropout, gradient-norm clipping at one, and batch size 128. Every fit receives 1,200 optimiser steps with sampling *with replacement*: 153,600 example presentations. Validation is recorded at step one and every 100 steps. Checkpoints and learning rates are chosen using within-range validation loss alone, separately for each seed. Test metrics are recorded for each fit's selected checkpoint; they do not choose its checkpoint or learning rate. The million-example follow-up uses a stricter delayed test phase.

Within-range integers are $[0, 131072)$; outside-range integers are $[131072, 262144)$. Each domain has separate validation and test sets of 4,096 and 8,192 values. All five splits are disjoint. The data seed is fixed at 1729, so the three reported seeds measure optimisation variability on one dataset, not variability across independently sampled datasets.

3 More data changes the answer, but not monotonically

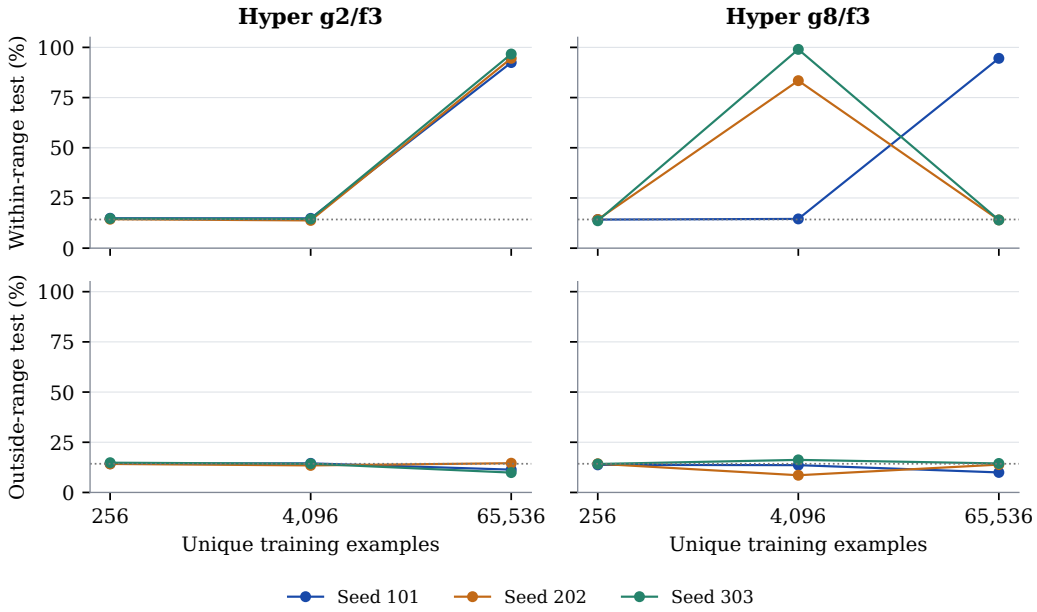


Figure 1: Completed primary screen. Each line is one optimisation seed; the two learning rates are selected by within-range validation for that seed. Dotted lines mark uniform chance, $1/7$. Training steps are fixed across data amounts. Points at different amounts are separate fits, not a continuation of one model.

For the shallow $g2/f3$ hypernetwork, increasing the pool to 65,536 examples produces a result that repeats across all three seeds: 92.44%, 94.56% and 96.69% within-range test accuracy. At 256 and 4,096 examples, its mean accuracy is 14.68% and 14.40%. There is a large change here, and three good seeds are much more useful evidence than the most flattering single run.

But the next hypernetwork makes the story less tidy. For $g8/f3$, 4,096 examples produce 14.56%, 83.42% and 99.01%; 65,536 produce 94.54%, 14.06% and 14.03%. So 4,096 can be enough for a successful fit. It was not reliably enough under this recipe, and adding data did not reliably rescue this deeper generator either.

There is an important budget detail behind the figure. Keeping 1,200 steps fixed means the nominal exposure is 600, 37.5 or 2.34375 passes through the three pools. The largest-pool runs encounter approximately 59,300 distinct training values over the full run, rather than every one of the 65,536. A selected checkpoint may be earlier still. We therefore cannot read this as a clean estimate of what data alone buys after each model has fully converged.

Scalar-input modulo stays near chance throughout the tested screen. Most binary architectures do too (Appendix A). This says something about these representations and recipes. It does not say that MLPs, transformers or relational networks are incapable of modular arithmetic. The plain 50-layer MLP also struggles on the smooth controls, while residual connections restore substantial learning. Making a stack deeper changes optimisation as well as its representational capacity.

4 The outside-range problem is doing real work

The successful shallow hypernetwork's 94.56% within-range score becomes 11.94% outside the range. That is below uniform chance on this particular held-out sample. A network that has learnt the general modular rule should not care that the integer is larger. Something in the fitted computation does.

There is a specific complication in the primary binary split: the eighteenth bit is always zero during training and always one outside. The model is being asked to handle a previously unseen feature value. To explore that issue we ran a separate diagnostic with 17 bits, training within $[0, 98304)$ and evaluating outside on $[98304, 131072)$. Every individual bit now varies during training. What remains unseen is the joint top-two-bit combination 11.

Table 1: Shallow hypernetwork $g2/f3$, 65,536 training examples. Test accuracy (percent), mean $\pm$ sample SD over three seeds. The diagnostic was declared after inspection of the first primary seed and is reported separately.

Domain design	Within range	Outside range
Primary: new high-bit value outside	94.56 ± 2.12	11.94 ± 2.39
Diagnostic: familiar bits, new joint pattern	81.40 ± 11.27	36.68 ± 28.79

The three diagnostic outside scores are 13.21%, 28.04% and 68.80%. There is useful extrapolation in some fits, but the variability is too large to call the behaviour dependable. And the diagnostic changes the range, bit width, data stream and training fraction together. It cannot isolate the causal contribution of exposing the high bit. We should resist giving it that job after the fact.

4.1 Is the hypernetwork taking a shortcut through a bias?

There is an obvious question about $f(x, g(x; \theta))$. Since the generator sees the input, perhaps it simply predicts the answer in the generated output bias and the rest of f is decorative. A frozen decomposition gives a useful first check. On the primary binary task at 65,536 examples, $g2/f3$ scores 14.73% $\pm$ 1.99% using only its final generated bias, compared with 94.56% $\pm$ 2.12% as the full model. Subtracting that bias from the full prediction leaves 90.69% $\pm$ 3.24%.

The final bias alone is not carrying the successful prediction. That is as far as this probe gets us. Other generated coefficients can carry input-dependent information, and removing a component after joint training is different from retraining without it. We have not identified a modular algorithm inside the network, or shown that a particular depth of f is necessary.

4.2 A note on the smooth controls

The broader screen helps keep the interpretation grounded. With 65,536 examples, a $g8/f1$ hypernetwork achieves outside-range $R^2 = 0.99626$ on multiplication and 0.93341 on sphere volume, averaged over seeds. Good extrapolation is possible on these finite domains. The chirp behaves very differently: some accurate within-range models produce very large outside errors. Whether a model fits a function and how it continues beyond the observations remain separate empirical questions, even before we get to modulo.

5 The million-example experiment

The follow-up keeps the full architecture comparison, expands the integer range, and gives every fit ten complete shuffled passes through one million unique training values. This is a different regime from the short screen. It changes range, exposure, bit width and regularisation together; it is not another point on Figure 1's data-size axis.

Table 2: Frozen follow-up design. Intervals are half-open. Validation and test values are disjoint from training and from one another.

Item	Setting
Within-range domain	$[0, 3145728)$
Outside-range domain	$[3145728, 4194304)$
Training / validation / test	1,000,000 training; 8,192 validation and 65,536 test per domain
Representation	Scalar and 22 binary features; every training bit varies
Grid	16 variants $\times$ 2 encodings $\times$ 2 learning rates $\times$ 3 decays $\times$ 3 seeds = 576 fits
Learning rates / weight decay	$0.0003, 0.003$ / $0, 0.01, 0.1$
Optimisation	AdamW; $\beta = (0.9, 0.999)$; $\epsilon = 10^{-8}$; constant learning rate
Other settings	Dropout 0; norm clipping 1; float32; TF32 disabled; decay applied to all learned parameters, including biases and normalisation
Exposure per fit	10 passes; 10,000,000 presentations; 80,000 updates
Logging	Every training batch; both validation domains every 5,000 presentations

The batch size is 128, with one small adjustment: $5000 = 39 \times 128 + 8$. Each validation boundary therefore finishes with an eight-example batch. This gives exactly the requested cadence without skipping or duplicating examples. Training loss in the figures is the example-weighted mean over those 5,000 presentations, not an unweighted mean that would give the eight-example batch disproportionate influence. Each fit has 2,000 validation observations.

Both validation sets are monitored, but **only within-range validation selects models**. Each fit retains the checkpoint with the lowest within-validation cross-entropy, using accuracy to break an exact tie and the earlier checkpoint if both tie. Once all 576 fits finish, we choose a learning-rate/weight-decay pair for each architecture and encoding by the mean best validation loss across its three seeds. We retain all three seeds of that configuration. An overall validation winner is also recorded for each encoding.

Only after those choices are frozen will the 96 retained models enter the single final test phase, on both held-out domains. The training worker receives no test file. The test evaluator uses a create-once execution record, and the selector rejects incomplete or mismatched sweeps. These controls protect the procedure; they do not make repeated validation observations independent evidence.

The outside range again holds out the top-two-bit pattern 11, while 00, 01 and 10 occur within range. This probes a new combination at fixed bit length. Even perfect performance here would not establish generalisation to arbitrary integer lengths or to every larger number.

6 Validation snapshot of the million-example sweep

This draft uses a fixed snapshot verified on **15 October 2026 at 16:48:57 UTC**. It contains 560 complete fits: all 288 binary fits and 272 of 288 scalar fits. The missing 16 are two scalar Transformer-8 fits and fourteen scalar Transformer-12 fits. Each included fit completed all ten passes. The counts below describe the snapshot; they do not freeze the final hyperparameter choices or report test scores.

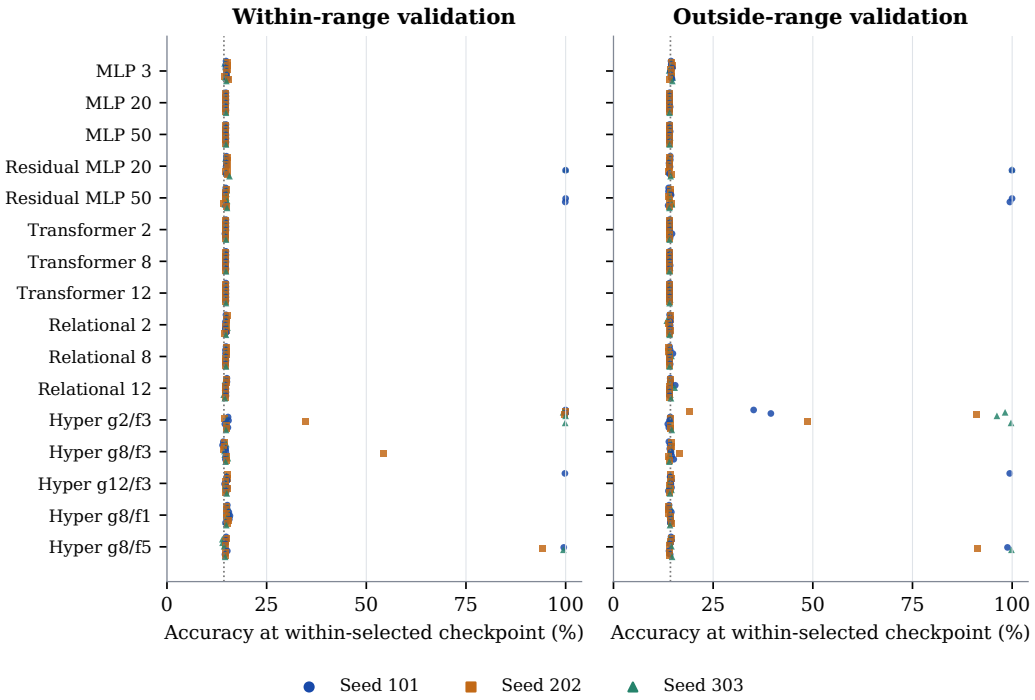


Figure 2: All 288 binary fits in the million-example snapshot: 18 per architecture, covering six learning-rate/decay settings and three seeds. Each point uses the checkpoint with minimum within-validation loss, including in the outside-range panel. Colours and shapes identify seeds; small vertical offsets separate settings and have no quantitative meaning. The dotted line marks $1/7$ accuracy. The two panels use the same fits, and neither uses test data.

Fourteen binary fits reach at least 99% within-range validation accuracy; seven reach that level on both validation ranges at the same checkpoint. The latter comprise two residual MLP-20 fits, two residual MLP-50 fits, and one each from $g2/f3$, $g12/f3$ and $g8/f5$. The 99% threshold is a post-hoc descriptive summary, not a selection criterion or a significance test. Figure 2 shows all scores, including those below it. None of the 96 binary settings (16 variants $\times$ six recipes) reaches this threshold on both ranges in all three seeds.

The scalar results occupy a much narrower range. Across the 272 completed fits, within-validation accuracy at the retained checkpoint is 14.20–15.86%, and cross-entropy is 1.94504–1.94572, close to $\log 7 \approx 1.94591$ for uniform prediction. These are results for this encoding, model budget and optimisation recipe. Sixteen scalar fits are still absent, and these observations do not establish that a scalar-input network cannot learn modulo arithmetic.

7 Learning trajectories and variation across seeds

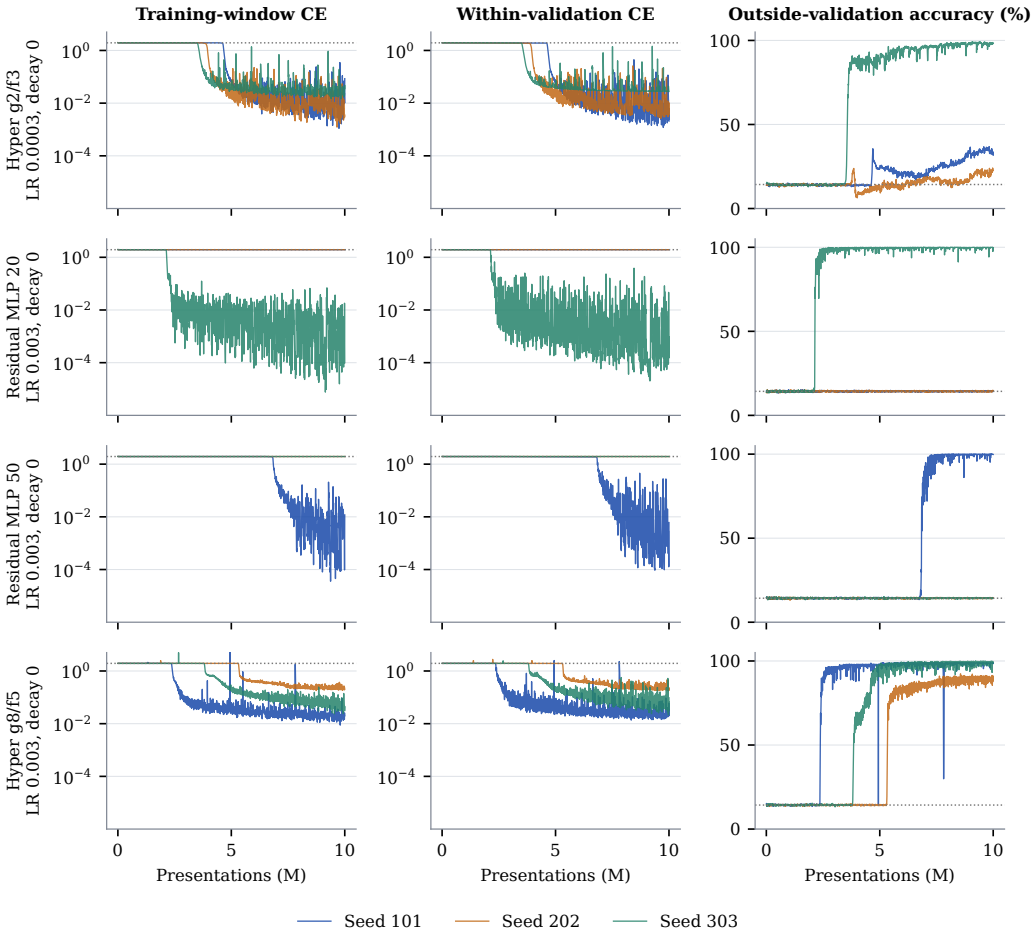


Figure 3: Four illustrative binary recipes, with all three seeds in each row. Recipes were chosen after inspecting the snapshot to show contrasting outcomes; they are not the final validation-selected configurations. All use zero decay. Training CE is the sample-weighted mean over each exact 5,000-presentation window; within CE and outside accuracy use the validation sets. Every recorded validation point is plotted. Loss axes are logarithmic; dotted lines mark $\log 7$ or $1/7$. Colours are consistent across panels.

The residual controls change the interpretation of the early hypernetwork results. At learning rate 0.003 and zero decay, one seed of each residual depth learns both ranges accurately while the other two remain near $\log 7$ within-validation loss. The successful seeds differ between depths: seed 303 for depth 20 and seed 101 for depth 50. High outside-range accuracy is therefore possible in this experiment without generating network weights per example. It remains unreliable across seeds.

7.1 A complete hyperparameter slice

Table 3: All 18 binary $g2/f3$ fits in the dated snapshot. Accuracy is percent at the within-loss-selected checkpoint, with outside accuracy taken at that same checkpoint. Seed columns are shown individually. No row is designated the final selected recipe.

		Within validation			Outside validation		
LR	Decay	101	202	303	101	202	303
0.0003	0	99.98	99.91	99.37	35.16	19.01	98.23
0.0003	0.01	99.93	99.78	99.98	39.48	91.10	96.18
0.0003	0.1	15.32	14.55	15.05	14.22	14.27	14.27
0.003	0	15.44	34.75	99.88	14.04	48.55	99.72
0.003	0.01	14.51	14.86	14.88	13.62	14.22	14.36
0.003	0.1	15.28	14.97	14.81	13.88	14.22	14.61

For binary $g2/f3$ at learning rate 0.0003 and zero decay, all three seeds exceed 99% within-range accuracy, while outside accuracy ranges from 19.01% to 98.23%. Adding decay 0.01 also produces three high within-range scores, with outside accuracies of 39.48%, 91.10% and 96.18%. At decay 0.1, the same learning rate leaves all three within-range scores below 16%. A single smooth "more regularisation helps" account does not describe this slice.

The higher learning rate changes which trajectories succeed. With zero decay, seed 303 reaches 99.88% within and 99.72% outside; the other two reach 15.44% and 34.75% within. Across this small grid, successful within-range fits can still have very different outside-range behaviour. The fixed test sets will assess the configurations selected from all three seeds, rather than promoting one successful seed as an architecture-level result.

7.2 What a late change does, and does not, establish

Some trajectories remain near uniform-prediction loss for a substantial part of training before improving sharply. This makes endpoint comparisons incomplete: two fits can look alike early and finish very differently under the same budget. The full curves retain every recorded 5,000-presentation observation supporting that statement.

These trajectories also require care with the term "grokking." The original report describes generalisation improving long after training performance has become good [4]. In the lower-learning-rate $g2/f3$ examples here, training-window and within-validation loss improve together during a late change. The window is measured on changing training batches rather than a fixed full training-set evaluation, so it is not itself a memorisation audit. We have not established the prolonged memorisation-then-generalisation separation needed for that interpretation.

8 What the evidence supports so far

The completed short screen shows that increasing the number of unique examples can change which architectures look useful under a fixed step budget. Its shallow conditional hypernetwork learns unseen within-range values with 65,536 examples, while the same configuration remains near chance at smaller amounts. The deeper generator's mixed seeds show why this cannot be reduced to a monotonic rule about data or depth.

The million-example snapshot broadens that picture. Several binary architectures can reach high accuracy outside the training interval, including residual MLPs. Most tested fits do not. Even recipes that reliably fit the within-range validation set can disagree substantially outside it. The stronger statement that one of these recipes is a dependable learner still needs the full selection procedure, the held-out tests and replication beyond the current three training seeds.

The next scientific question is what the successful computations have in common. For example, do accurate outside-range fits implement the repeating binary weights $1, 2, 4$, or a different finite-domain approximation? The present measurements do not identify the learned algorithm. Retrained generator/component controls, independent dataset seeds and matched domain shifts could distinguish those possibilities. These are proposed follow-ups, not results from the current sweep.

The remaining work under the present protocol is to complete the 16 missing fits, freeze the choices using within-validation loss across all three seeds, and evaluate the 96 retained checkpoints once on both test sets. The final report will replace this dated snapshot with those results while retaining this version's evidence.

Scope and limitations. Three optimisation seeds on one fixed dataset measure limited training variation; they are not independent samples of the data-generating split. The 576 fits are not 576 independent replications of one method. Parameter matching changes width with depth and does not match FLOPs. Two learning rates and three decay values are a bounded search, not evidence that every family has been optimally tuned. Repeated checkpoint selection can overfit validation. The 99% counts and the illustrative trajectory choices are descriptive decisions made after viewing the results. The original screen, observed-bit diagnostic and longer follow-up change several factors and must remain separate comparisons. None tests variable-length arithmetic, negative integers, formal algorithm discovery or an unbounded domain.

Authorship and execution. The research question, requested architecture families and follow-up choices are Antreas's. Protocol development, implementation, analysis and report preparation were assisted by Codex under his direction. This draft has undergone computational checks and self-review; it has not received independent scientific review or Antreas's approval of the revised prose. It is not a peer-reviewed publication.

A All architecture outcomes in the completed screen

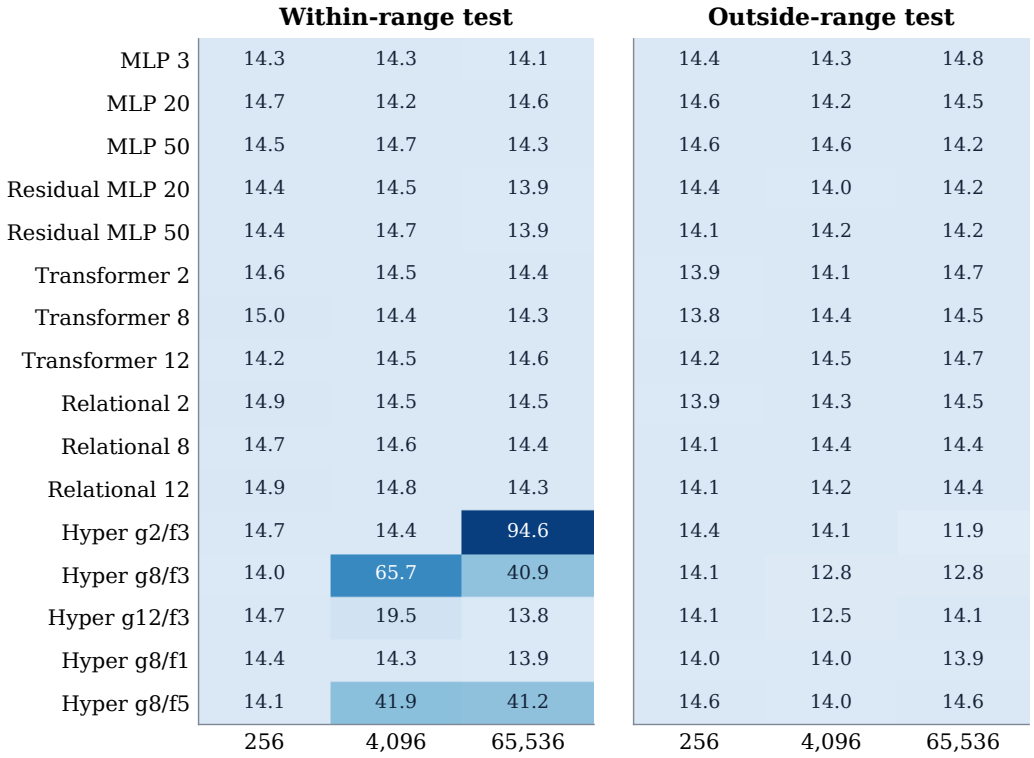


Figure 4: Binary modulo 7, all 16 variants and three data amounts. Entries are mean test accuracy (percent) over the three validation-selected seeds. The underlying seed rows, standard deviations, losses, validation metrics and both learning-rate fits are included in the public data. Means can conceal very mixed seed outcomes; Figure 1 shows two examples.

Table 4: The 65,536-example slice, percent accuracy, mean ± sample SD. The scalar outside results are also near chance and are available in the CSV.

Architecture	Scalar within	Binary within	Binary outside
MLP 3	14.45 ± 0.17	14.05 ± 0.25	14.82 ± 0.13
MLP 20	14.55 ± 0.11	14.55 ± 0.11	14.47 ± 0.33
MLP 50	14.34 ± 0.44	14.34 ± 0.44	14.25 ± 0.36
Residual MLP 20	13.98 ± 0.37	13.92 ± 0.03	14.15 ± 0.22
Residual MLP 50	14.25 ± 0.10	13.88 ± 0.09	14.19 ± 0.25
Transformer 2	14.42 ± 0.11	14.43 ± 0.06	14.67 ± 0.06
Transformer 8	14.45 ± 0.06	14.34 ± 0.24	14.48 ± 0.31
Transformer 12	14.63 ± 0.19	14.62 ± 0.23	14.71 ± 0.09
Relational 2	14.22 ± 0.59	14.47 ± 0.51	14.49 ± 0.33
Relational 8	14.47 ± 0.26	14.36 ± 0.14	14.37 ± 0.30
Relational 12	14.29 ± 0.23	14.29 ± 0.43	14.44 ± 0.15
Hyper g2/f3	14.12 ± 0.33	94.56 ± 2.12	11.94 ± 2.39
Hyper g8/f3	14.10 ± 0.49	40.88 ± 46.48	12.76 ± 2.43
Hyper g12/f3	14.14 ± 0.07	13.83 ± 0.32	14.14 ± 0.50
Hyper g8/f1	14.44 ± 0.34	13.91 ± 0.42	13.85 ± 1.08
Hyper g8/f5	14.51 ± 0.15	41.15 ± 46.79	14.58 ± 0.72

B Data, checks and reproduction

The public repository includes all 1,728 primary fits and all 96 diagnostic fits, their recorded learning curves, per-seed selected results and summary tables. The modulo subset of the primary screen comprises 576 fits. The original twelve-fit million-example export remains available with its 24,000 validation observations and 960,000 individual training batches. The new `draft-2026-10-15` directory adds 560 fit summaries, all 192 planned three-seed recipe cells (with incomplete means left blank), and 24,000 full-resolution validation observations for the twelve illustrative trajectories. The dated v0.1.0 release is preserved. This revised draft is prepared locally and is not yet the final public release.

The completed-screen audit verified 1,840 stored artifacts by exact generation and SHA-256, recomputed 7,296 evaluation sets, and replayed 912 selected checkpoints on 16 examples per evaluation split. That was a bounded check, not independent retraining. Checkpoint and per-example prediction payloads are not part of the initial public release; their hashes are retained in the exported fit records.

For the 560-fit follow-up snapshot, the private metric audit covers 56,007 linked segments, 44,800,000 training-batch records and 1,120,000 validation records. Each complete history covers presentations 0 through 10,000,000 without gaps or double-counting replayed work. The audit checks finite values, exact sample and step boundaries, selected checkpoints, and the sample-weighted training windows. The draft exporter rechecks all 560 compressed validation-history hashes and recomputes each within-selected checkpoint from its 2,000 observations. Outside scores are joined to that checkpoint, never independently maximised. The complete large training histories remain in the research archive pending final asset packaging; they are not all included in this compact draft export.

Public checks reproduce the descriptive counts, three-seed means and sample standard deviations, the shown trajectories, and the retained early-snapshot training-window calculations. The protocol suite additionally exercises split separation, scalar encoding uniqueness, all 32 architecture/encoding forward and backward paths, exact sample boundaries, validation-only selection and deterministic checkpoint resumption on a small CPU case. Checks establish specific consistency properties; they are not an independent retraining or a causal explanation of the observed differences. No million-example test data are used in this draft.

Source and environment. Source commits and byte hashes identify the exported scientific modules: `68c142af3f3e` for primary training, `78d417c1bae3` for the diagnostic, and `de2c10e63431` for the million-example sweep. Scientific modules are unchanged; the public storage adapter and entry point support local reproduction. GPU runs used PyTorch 2.8.0, CUDA 12.8, NumPy 2.3.2 and Python 3.13.13. CPU reproduction can differ numerically.

Resource measurement. The completed primary screen and diagnostic used 1.62639 allocated GPU-hours, including setup failures, with one GPU at a time. This excludes invoiced cost and virtual-machine overhead. Final usage for the million-example sweep is pending.

Reading the repository. See `PROTOCOL.md` for the full splits and selection rules. Rebuild figures with `analysis/make_figures.py` and edit `report/report.tex`. The official NeurIPS 2026 template is hash-pinned and used in preprint mode without a conference affiliation claim.

References

[1] David Ha, Andrew Dai and Quoc V. Le. *HyperNetworks*. 2016. arXiv:1609.09106.

[2] Adam Santoro, David Raposo, David G. T. Barrett, Mateusz Malinowski, Razvan Pascanu, Peter Battaglia and Timothy Lillicrap. *A simple neural network module for relational reasoning*. 2017. arXiv:1706.01427.

[3] Ilya Loshchilov and Frank Hutter. *Decoupled Weight Decay Regularization*. 2017. arXiv:1711.05101.

[4] Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin and Vedant Misra. *Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets*. 2022. arXiv:2201.02177.